

Жизненный цикл: подключение, биллинг, долги

Информационная система городской телефонной сети

Как заявка превращается в линию, как считаются деньги и кто что делает

Содержание

1	О чём этот гайд	2
2	Жизненный цикл одной линии (обзор)	2
3	Этап 1–2. Регистрация и заявка (абонент)	2
4	Этап 3. Подключение оператором	3
5	Этап 4. Пользование линией (абонент)	3
	5.1 Межгород: включить/выключить	3
	5.2 Звонки	4
6	Этап 5. Биллинг: как считаются деньги	4
	6.1 Тариф и абонплата	4
	6.2 Виды счетов и долг	4
	6.3 Оплата	5
7	Что считает БД сама, а что — оператор руками	5
8	Параметры биллинга (billing_settings)	5
9	Связь с аналитикой (откуда оператор узнаёт, что пора действовать)	6
10	Сквозной сценарий (демо «запуск в городе»)	6
11	Шпаргалка к защите (по операциям)	6

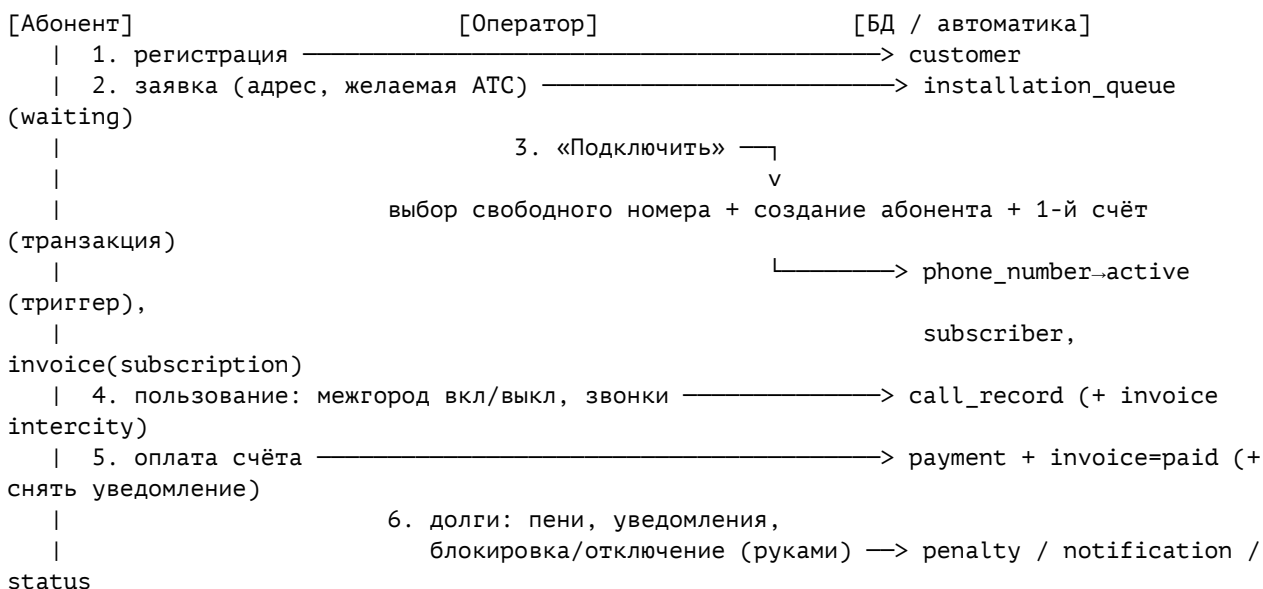
1 О чём этот гайд

Остальные документы объясняют **устройство** (таблицы, запросы, CRUD, авторизацию). Здесь — **движение**: как данные «оживают» в рабочем сценарии. Три действующих лица:

Кто	Что делает
Абонент (личный кабинет)	регистрируется, подаёт заявку, управляет линией, звонит, платит
Оператор (панель)	подключает заявки, ведёт справочники, начисляет/закрывает долги
База данных	сама держит целостность (триггеры), считает абонплату и долги (функция/представления)

Где смотреть. Бизнес-логика живёт в трёх местах: backend/server/src/ops.rs (подключение оператором), portal.rs (действия абонента), а расчёты — в migrations/0009_views.sql (функция fn_subscriber_monthly_fee, представление v_subscriber_debt) и migrations/0005_billing.sql (тарифы, настройки).

2 Жизненный цикл одной линии (обзор)



Этапы 1–5 автоматизированы кодом, этап 6 — **ручные** операторские действия (см. раздел 7).

3 Этап 1–2. Регистрация и заявка (абонент)

Регистрация (POST /api/portal/register) заводит строку customer и сразу открывает сессию. Затем абонент подаёт заявку:

```
// portal.rs :: apply - POST /api/portal/applications
// 1) создаём адрес заявителя
INSERT INTO address (postal_index, district, street, house, apartment) ... RETURNING id;
// 2) тип очереди — по категории абонента
let queue_type = if customer.category == "privileged" { "privileged" } else { "regular" };
// 3) сама заявка, привязана к аккаунту
INSERT INTO installation_queue
  (applicant_*, queue_type, address_id, desired_pbx_id, customer_id)
VALUES (... , 'waiting'-по-умолчанию);
```

- Льготники попадают в **льготную** очередь (queue_type='privileged') — это требование задания о приоритете.
- Заявка хранит **желаемую** АТС (desired_pbx_id), но финальное слово — за оператором.

- `customer_id` связывает заявку с аккаунтом: позже из него возьмут ФИО/категорию для абонента.

4 Этап 3. Подключение оператором

Оператор видит очередь (`ApplicationsView.vue`), выбирает АТС и тип линии и жмёт «Подключить» → `POST /api/ops/applications/{id}/provision` (право `queue:update`). Вся операция — в **одной транзакции**, чтобы не получить «полужабку».

```
// ops.rs :: provision (упрощённо)
// проверки: заявка есть, не 'installed', есть customer_id, выбрана АТС
let number_id = /* SELECT id FROM phone_number
                  WHERE pbx_id = $1 AND status = 'free' ORDER BY number LIMIT 1 */; //
нет → 400
let intercity = if pbx_type == "city" { "closed" } else { "none" }; // межгород есть
только у городских

BEGIN;
UPDATE phone_number SET line_type=$, intercity=$, address_id=$ WHERE id = number_id;
INSERT INTO subscriber (... , phone_number_id, address_id, customer_id, connected_at)
SELECT c.last_name, c.first_name, ... FROM customer c WHERE c.id = customer_id; --
данные из аккаунта
INSERT INTO invoice (subscriber_id, kind='subscription', period=тек.месяц,
                    amount = fn_subscriber_monthly_fee(subscriber_id),
                    due_date = 20-e, status='pending')
ON CONFLICT (subscriber_id, kind, year, month) DO NOTHING; --
первый счёт
UPDATE installation_queue SET status='installed', assigned_number_id = number_id WHERE id
= $;
COMMIT;
```

Что происходит по шагам:

1. **Берётся свободный номер** на выбранной АТС (самый младший по порядку). Нет свободных → 400 «на выбранной АТС нет свободных номеров».
2. **Номеру задаются** тип линии, межгород (`closed` у городской — доступен, но выключен; `none` у замкнутых сетей) и адрес установки.
3. **Создаётся абонент** — ФИО, пол, дата рождения, категория, льгота **копируются из аккаунта** `customer`; `connected_at` = сегодня.
4. **Выставляется первый счёт** абонплаты за текущий месяц на сумму `fn_subscriber_monthly_fee` со сроком 20-го числа.
5. **Заявка закрывается**: `status='installed'`, в неё прописывается выданный номер.

БД делает сама. Как только абонент привязан к номеру, **триггер** `number_status_sync (0008)` сам переводит `phone_number.status` из `free` в `active` — статус не выставляют руками.

5 Этап 4. Пользование линией (абонент)

Перед любым действием с линией портал проверяет **владение** (`owned_subscriber` → 403, если линия чужая).

5.1 Межгород: включить/выключить

```
// portal.rs :: set_intercity - PUT /api/portal/lines/{id}/intercity
if current_intercity == "none" { return 400 "межгород недоступен (замкнутая сеть)"; }
UPDATE phone_number SET intercity = (if enabled {'open'} else {'closed'});
```

- Доступно **только** на городских АТС (где `intercity` не `none`).
- Включение (`open`) поднимает абонплату: функция `fn_subscriber_monthly_fee` берёт тариф с `with_intercity = true` (см. этап 5).

5.2 Звонки

```
// portal.rs :: make_call - POST /api/portal/lines/{id}/call
duration = clamp(1, 36000);
// местный:
if kind == "local" { /* найти dest_number; */ INSERT call_record(call_type='local',
cost=0); }
// межгород:
if kind == "intercity" {
  if intercity != "open" { return 400 "межгород закрыт"; }
  minutes = ceil(duration/60); cost = minutes * 5.0; // 5 руб/мин
  INSERT call_record(call_type='intercity', dest_city_id, cost);
  INSERT INTO invoice (... kind='intercity', тек.месяц, amount=cost, due 20-е, 'pending')
    ON CONFLICT (...) DO UPDATE SET amount = invoice.amount + EXCLUDED.amount,
status='pending';
}
```

- **Местный** звонок — бесплатный (cost = 0), просто запись в журнал (call_record).
- **Межгород** — требует включённой услуги; стоит 5 руб за начатую минуту; каждый такой звонок **докидывает** сумму в **единый счёт за межгород** текущего месяца (DO UPDATE суммирует). Так за месяц копится один счёт intercity, а не десяток.

6 Этап 5. Биллинг: как считаются деньги

6.1 Тариф и абонплата

Абонплата не хранится у абонента — её **вычисляет функция** по тарифу:

```
-- 0009 :: fn_subscriber_monthly_fee(subscriber_id)
-- тариф = (тип линии, открыт ли межгород); льготнику — скидка из настроек
SELECT monthly_fee FROM tariff
WHERE line_type = v_line_type AND with_intercity = (pn.intercity = 'open')
ORDER BY valid_from DESC LIMIT 1;
IF category = 'privileged' THEN fee := round(fee * (1 - privilege_discount), 2); END IF;
-- по умолч. 50%
```

- Цена зависит от **типа линии** (main/parallel/paired) × **наличия межгорода**. Тарифы — в таблице tariff (0010 заливает 6 строк).
- **Льготникам** — скидка privilege_discount из billing_settings (по умолчанию 0.5 = 50%).
- Функция помечена STABLE и вызывается «на лету» — в кабинете (monthly_fee в обзоре) и при выставлении счёта. Отдельно сумму абонплаты нигде не дублируют.

6.2 Виды счетов и долг

Сущность	Смысл
invoice (subscription)	счёт за абонплату месяца; один на (абонент, вид, год, месяц)
invoice (intercity)	накопленный счёт за межгород месяца
payment	факт оплаты конкретного счёта
penalty	пени за просрочку (накапливается, флаг paid)
notification	письменное уведомление о долге с deadline

Долг считает **представление** v_subscriber_debt — суммирует неоплаченные счета и пени, разбивая на абонплату / межгород / пени:

```
-- 0009 :: v_subscriber_debt (суть)
subscription_debt = Σ invoice(kind='subscription', status IN ('pending', 'overdue'))
intercity_debt    = Σ invoice(kind='intercity',      status IN ('pending', 'overdue'))
penalty_debt      = Σ penalty(NOT paid)
total_debt        = сумма трёх; oldest_due_date = самый ранний срок
```

6.3 Оплата

```
// portal.rs :: pay_invoice — POST /api/portal/invoices/{id}/pay
// проверки: счёт принадлежит абоненту; ещё не 'paid'
BEGIN;
  INSERT INTO payment (subscriber_id, invoice_id, amount);
  UPDATE invoice SET status = 'paid' WHERE id = $;
  UPDATE notification SET resolved = TRUE -- снять уведомление того
же вида
  WHERE subscriber_id = $ AND kind = (subscription_debt|intercity_debt) AND NOT resolved;
COMMIT;
```

Оплата — тоже транзакция: создаётся payment, счёт становится paid, и **снимается** связанное непогашенное уведомление о долге.

7 Что считает БД сама, а что — оператор руками

Это ключевой момент для понимания проекта.

БД делает сама. Автоматически (триггеры/функции/представления, без участия человека):

- целостность данных — триггеры 0008 (тип АТС, межгород, лимит абонентов на номер, один дом, синхронизация статуса номера);
- статус номера free ↔ active при появлении/уходе абонента;
- расчёт абонплаты (fn_subscriber_monthly_fee) и суммы долга (v_subscriber_debt) — по запросу.

БД делает сама. Создаётся действиями приложения (в коде, в транзакциях):

- первый счёт абонплаты — при подключении (provision);
- счёт за межгород — при междугороднем звонке (накопительно за месяц);
- запись звонка; оплата (payment + invoice=paid + снятие уведомления).

Делается вручную. Нет планировщика (cron/фоновых задач) — поэтому делается **вручную** оператором через обычный CRUD, опираясь на аналитику:

- начисление абонплаты за **следующие** месяцы (повторные счета);
- перевод просроченных счетов в overdue;
- начисление **пеней** (penalty_daily_rate в настройках есть, но автоматического расчёта нет);
- отправка **уведомлений** (notification с дедлайном);
- **блокировка межгорода** / **отключение** должника (subscriber.status, phone_number.status='blocked') и повторное подключение (reconnection_fee).

Важно. Для защиты честная формулировка: модель биллинга **полная** (тарифы, скидки, пени, уведомления, отключения заложены в схему и настройки), но **начисление по расписанию не автоматизировано** — это сознательная граница учебного проекта. Операторские решения принимаются по данным аналитики.

8 Параметры биллинга (billing_settings)

Одна строка-одиночка (id=1), которую правит оператор на странице «Настройки»:

Параметр	По умолч.	Смысл
privilege_discount	0.500	скидка льготнику на абонплату (50%)
reconnection_fee	150.00	плата за повторное подключение после отключения
penalty_daily_rate	0.00100	дневная ставка пени (0.1% от долга)
payment_due_day	20	день месяца — срок оплаты счёта

Параметр	По умолч.	Смысл
notice_grace_days	2	запас дней в уведомлении сверх срока

Важно. Срок оплаты (20-е число) в коде provision/make_call сейчас **зашит константой**, совпадающей со значением payment_due_day по умолчанию. Если менять день в настройках — это место стоит начать читать из billing_settings.

9 Связь с аналитикой (откуда оператор узнаёт, что пора действовать)

Ручные шаги этапа 6 опираются на запросы варианта (см. docs/queries.pdf):

Запрос	Зачем оператору
Q3. Должники	кому выставлять пени/уведомления
Q4. Рейтинг АТС по долгам	где сосредоточен долг
Q13. Должники, требующие действий	кого пора уведомлять/отключать
Q2. Свободные номера	есть ли что выдать при подключении

10 Сквозной сценарий (демо «запуск в городе»)

1. /portal: регистрация (Иванов) -> customer
2. /portal/apply: адрес + желаемая АТС -> installation_queue (waiting)
3. /staff: вход admin/admin -> «Заявки» -> Подключить (АТС, main)
-> phone_number(active), subscriber, invoice(subscription, pending, 20-e)
4. /portal: линия видна; включить межгород -> phone_number.intercity=open
(абонплата выросла)
5. /portal: звонок в другой город 3 мин -> call_record(intercity, 15 ₺) +
invoice(intercity)
6. /portal: оплатить счёт -> payment + invoice=paid
7. (оператор, при долге) пеня/уведомление/блок -> penalty / notification / status
[вручную]

11 Шпаргалка к защите (по операциям)

Вопрос	Короткий ответ + где код
Как заявка становится линией?	ops.rs::provision: свободный номер → абонент из ак-каунта → первый счёт → заявка installed. Всё в одной транзакции.
Откуда берётся номер?	Самый младший free на выбранной АТС; нет — 400. Статус → active триггером.
Как считается абонплата?	fn_subscriber_monthly_fee: тариф (тип × межгород), льготнику -50%. 0009.
Как тарифицируется межгород?	5 ₺ за начатую минуту; копится в один месячный счёт intercity (DO UPDATE). portal.rs::make_call.
Как считается долг?	Представление v_subscriber_debt: неоплаченные счета (абонплата+межгород) + пени. 0009.
Что при оплате?	payment + invoice=paid + снятие уведомления, в транзакции. portal.rs::pay_invoice.

Вопрос	Короткий ответ + где код
Что автоматизировано, что — нет?	БД: целостность, статус номера, расчёты. Вручную: повторные счета, пени, уведомления, отключения (нет планировщика).
Где абонент защищён от чужого?	owned_subscriber → 403; перед каждым действием с линией. portal.rs.

См. также: запросы — docs/queries.pdf, авторизация — docs/auth.pdf, миграции — docs/migrations.pdf, схема — docs/schema.dbml.